

PROOF OF CONCEPT

Lot Genius

Product Requirements Document

Prepared for: Steffes Group, Inc.

Prepared by: Kadima Consulting · Philippe Richard

Document: Product Requirements Document (PRD)

Version: v1.0 (Draft)

Date: 11 June 2026

Classification: Confidential

KADIMA // CONSULTING

Agentic AI systems · Trustworthy, governed, self-improving intelligence

Lot Genius — Proof of Concept

Product Requirements Document (PRD)

Project	Lot Genius PoC — Agentic auction-data agent modernization
Client	Steffes Group, Inc. <i>(verify exact legal entity string before any binding doc)</i>
Supplier	Kadima Consulting
Author	Philippe Richard (Kadima Consulting)
Document version	v1.0 (draft)
Date	2026-06-11
Governing SoW	Lot Genius Proof of Concept — Time & Material, ~20 hours, ~1 week
Status	DRAFT for review — contains items marked TBD requiring client / counsel confirmation

Internal vs client-shareable. This is a combined document. Sections are tagged: - **CLIENT-SHAREABLE** — safe to share with Steffes. - **INTERNAL/BUILD** — Kadima build detail (permissions, IP mechanics); summarize, do not hand over verbatim.

Section 10 (Phase 2) is written at "brochure altitude" and is client-shareable; it discloses **no** Kadima IP mechanics.

1. Problem & Current State

1.1 What Lot Genius is

Lot Genius is an internal Steffes Group AI agent that lets ~10–20 auction appraisers ask natural-language questions about historical auction data and get conversational answers backed by real data — surfaced inside Microsoft Teams and Microsoft 365 Copilot. It is a chat agent, not a dashboard. Typical questions:

- "Show me 5 comps for a 2023 John Deere X9 1100."
- "What's the year-over-year hammer-price trend for combines?"
- "Comps for a 2018 MFWD tractor in Iowa."

1.2 Current architecture (works, but brittle)

```
Teams / M365 Copilot
  → Copilot Studio agent
    → custom connector (Power Apps, single shared API key)
      → Azure Function (/api/ask) → gpt-4.1-mini GENERATES SQL
        → Synapse Serverless SQL (curated views; ~500K-row Lot table, 481 cols)
```

An admin web console tunes behavior (category include/exclude, make-model aliases, PII column blocklist, schema visibility, prompts, few-shots via Blob + GitHub PRs).

1.3 Pain points this PoC targets

#	Pain	Root cause
P1	Brittle answers, 30s connector timeouts	LLM-generated SQL is fragile and slow
P2	Fabricated values	Model invents numbers when data is missing
P3	No real "comps" / fuzzy matching	No semantic / vector retrieval — only exact SQL
P4	Coarse PII handling	Column blocklist only; no value/intent-level redaction
P5	Context-switching failures	Limited context window in Copilot Studio orchestration
P6	Licensing / access friction	PAYG Copilot Studio credits; user license edge cases
P7	Platform end-of-life risk	Microsoft is deprecating Synapse ("Long Haul"), pushing Fabric

1.4 What this PoC does NOT change

Per the SoW, **Synapse Serverless remains the system of record** for the PoC. Fabric migration is future-roadmap guidance only (see §10 and §11). The PoC modernizes the *agentic and retrieval* layer, not the data platform.

2. PoC Hypothesis & Success Criteria

2.1 The hypothesis this PoC exists to validate

An agentic, multi-agent retrieval system, grounded in semantic comps search over a real Lot subset, can answer an appraiser's natural-language auction questions inside Teams — faster, more accurately, and with less fabrication than the current LLM-generates-SQL pipeline — while keeping a human in the loop on classification.

Center of gravity: **"comps that are actually comparable, without fabrication."**

2.2 Success criteria

Measured against a **frozen evaluation set** of ~20–30 gold questions with human-blessed answers, built in the discovery workshop (see §12). *No eval set → no success claim.*

ID	Criterion	Target	Why it is the bar
SC1	Comps relevance — appraiser rates returned comps "usable"	≥ 80% on the gold set	The capability the current system cannot do at all.
SC2	Fabrication rate — any price/spec/date not traceable to a source row	≤ 2%; every number cites a Lot ID	Don't ship the current system's disease in a new body.
SC3	Latency under the wall — end-to-end question→answer in Teams	p50 ≤ 10s, p95 < 25s	Must clear the 30s connector timeout with margin.
SC4	Admin-edit-to-live-effect — correct a classification, re-ask, see it fixed	Same session, < 5 min, no redeploy	Makes the human-editable + self-improvement directive real and demonstrable.

Explicitly NOT success criteria for this PoC: throughput, concurrent-user load, cost-per-query optimization, full-corpus coverage. These are production concerns (out of SoW scope).

3. Users & Priority User Stories

Audience	Size	Surface	Use
Appraisers (primary)	~10–20	Teams chat, M365 Copilot agent picker	Comps, valuation, description scoring, trend analysis
Operations / Admin	1–2	Admin web console	Tune categories/aliases/PII, review feedback, edit classifications
External / public	—	Not exposed	Tenant-internal only

Priority user stories (demo must hit P0s):

- **P0 — Appraiser (money shot):** Ask "show me 5 comps for a 2023 John Deere X9 1100" in Teams → 5 genuinely comparable lots with hammer prices, each citing its source Lot ID, in under 10 seconds. (SC1, SC2, SC3)
- **P0 — Appraiser (trust):** When the system can't find good comps or isn't sure, it says so instead of inventing numbers. (SC2)
- **P0 — Admin (directive made visible):** See how a lot/query was classified, correct it in the console, re-ask → corrected result, same session, no redeploy. (SC4) — **primary anchor = equipment categorization.**
- **P1 — Admin (PII secondary anchor):** Mark a column/value sensitive and see it redacted on the next query (thin secondary demo if hours allow).
- **P1 — Appraiser:** YoY hammer-price trend for combines, with the row count behind the aggregate.
- **P1 — Admin:** See which queries returned low-confidence / low-relevance results (read-only — the visible seam of the self-improvement loop).
- **P2 — cut-first:** multi-turn context retention ("now show only the ones under \$400k").

4. Scope

The only way to honor three ambitious directives (agentic-AI-framework substrate, surface+edit+manage classification, agentic RAG with incremental improvement) in ~20 hours is the discipline: **seams are real and demonstrated on a narrow slice; scale and autonomy are documented as roadmap.**

4.1 In scope — BUILD (the spine)

- Vector index + embedding pipeline over a **representative Lot subset** (e.g. combines + tractors — deep enough for real comps, not all 500K rows).
- The 5-agent chain (orchestrator + retrieval + PII + reasoning + formatter) running **on Kadima's agentic AI framework** behind a single MCP server.
- Hybrid retrieval: pgvector semantic comps + parameterized Synapse SQL for exact aggregates.
- PII redaction **demonstrably better than a column blocklist** on the demo slice.
- One live Teams / M365 Copilot appraiser path; one admin-console screen.
- The **classification receipt UI** (what was included/excluded and why) with per-line edit.
- A **minimal ReasoningBank** (store episode → reuse on next similar query).
- 1-page target architecture + Fabric-migration roadmap; knowledge-transfer; final report.

4.2 Architected-for, demonstrated-thin (shown as a working seam on a slice; scale documented)

- **Self-improvement engine** — build the *plumbing* (corrections persist, retrieval reads corrected store next query). What is *demonstrated* = human-correction-persists-and-takes-effect (SC4). What is *architected* (autonomous re-classification, gain-controlled trust, batch re-embedding) = roadmap.
- **Agentic AI framework as substrate** — the SoW's 5 agents are genuinely built on Kadima's agentic AI framework; the broader ~40-agent framework is the engine, not a PoC deliverable.
- **Per-user memory partitions** — partition key designed; demoed with one shared bank.

4.3 Out of scope (per SoW — hold the line)

- Full Fabric / Lakehouse migration.
- MachineryPete and other external-source expansion.
- Per-user reasoning banks at production depth.
- Production-scale performance/cost optimization.
- Per-user **row-level** data authorization (**decision: permissive for the PoC** — see §8).

5. Target Architecture (build detail where noted)

5.1 Runtime path

```
Teams / M365 Copilot
└─> Foundry Agent Service: ORCHESTRATOR (thin; intent-classify + tool-calling only)
    |   model: small/fast Foundry chat model (intent routing)
    └─> ONE MCP SERVER ← THE SEAM (Azure Container App; Kadima agentic AI framework)
        └─ tool: comps_search → pgvector (Postgres Flexible Server)
        └─ tool: structured_query → Synapse Serverless (PARAMETERIZED SQL templates)
        └─ tool: pii_scrub → PII/Security agent
        └─ tool: analyze → Reasoning agent + ReasoningBank
            model: MAI-Thinking-1 (valuation reasoning + validation gate)
```

Core principle: "vector finds the lots, SQL supplies the trusted numbers." The LLM only ranks and narrates rows the database returned — it never invents a price. This single discipline removes most of P1/P2.

The MCP server is the architectural seam AND the IP boundary (see §9). The framework internals (ReasoningBank, the agent swarm, memory) live *behind* it; Foundry sees typed tools, not a swarm.

5.2 Retrieval split

- **pgvector gets:** embeddings of a denormalized lot text blob per lot — `normalized make/model + title + description + key spec columns` — plus a metadata sidecar (`lot_id, category, sale_date, price, geography`) in the same row for filterable hybrid search.
- **Synapse keeps:** exact aggregates, trends, geography filters, counts/averages/percentiles — as **5–8 parameterized templates** (e.g. `avg_price_by_category(category, date_range, region)`), **never free-form generated SQL**.
- **Orchestrator routing (deliberately simple):** "comparable lots / what did things like X sell for" → `comps_search`; "average / trend / how many / by region" → `structured_query`; mixed → call both and fuse.

- **Fusion:** pgvector returns candidate `lot_id`s + similarity; those IDs feed a structured query that pulls authoritative price/date/region from Synapse. Vector finds; SQL supplies the numbers.

5.3 Models & SDKs

Component	Model	Notes
<code>analyze</code> (reasoning + validation gate)	MAI-Thinking-1	Flagship reasoning model on Foundry; lives behind the seam. One heavy model per request path.
Orchestrator intent routing	small/fast Foundry chat model (e.g. <code>gpt-4o-mini</code> -class)	A 3-way branch does not need a flagship model.
Embeddings for pgvector	Azure OpenAI <code>text-embedding-3-large</code> (3072-dim)	No MAI model embeds — this fills the gap. Dimensionality is a workshop knob; embedding source is immutable for the life of the index.
Build-time coding assistant	MAI-Code-1-Flash	Build-time ONLY (scaffold Container App, SQL templates, MCP stubs in Copilot/VS Code). Never wired to generate SQL at runtime.

Microsoft SDKs: - **Foundry Agent Service SDK** — *client side*. Builds the thin orchestrator (agent definition, tool-binding, intent routing, MI auth). This is what Steffes receives and can re-host. - **MCP SDK** — *on the seam*. Defines + serves the four tool contracts. Schema is client-facing; implementation is Kadima's. - **Microsoft Agent Framework** (converged Semantic Kernel + AutoGen successor) — *Kadima side, behind the seam, optional*. Only stand it up if `analyze` needs in-tool agent fan-out beyond what the agentic AI framework provides.

Microsoft Frontier Tuning ("trained on your data, in your environment") = **roadmap only** (see §10/§11). Fine-tuning collapses auditability, bakes a data snapshot into weights, and burns the budget on a non-deterministic artifact. The PoC gets "learns from your data" from inference-time memory (ReasoningBank), with full provenance and zero retrain.

6. Agentic-RAG: Classification + Self-Improvement Loop

Classification is the chokepoint — get it wrong and exact SQL aggregates over the wrong category, which is *confidently* wrong. It is treated as a **constraint-propagation gate**, not a freeform LLM step.

```

query
├─[1] ReasoningBank lookup (seen a query like this?) → hit: reuse mapping (deterministic, fast)
├─[2] Classifier: pgvector proximity → candidate categories
|   + admin overrides (include/exclude, aliases) applied as HARD constraints
├─[3] SURFACE a classification RECEIPT (chosen path, normalization, included/excluded + reasons,
|   confidence with decomposition, Edit affordance per line)
├─[4] VALIDATION GATE: constraints satisfied? confidence above floor? blocklist clean?
|   fail / low-conf → escalate to human BEFORE acting (no silent guessing)
└─[5] On correction OR success → write episode to ReasoningBank (gain-weighted trust)

```

The invariant: reasoning (1–3) is separated from action (the SQL / comp execution) by a validation gate (4). A classification is a *bet* until a human or constraint check confirms it; only then does it touch data. This is what makes it auditable.

6.1 Classification — three layers, hard constraints first

1. **Alias / override exact match (hard).** Admin-curated rules are hard constraints; "tractor" → if an alias maps it, that wins deterministically (no vector math). This is the determinism anchor.
2. **pgvector proximity (soft).** For uncovered terms, embed the query and rank candidate categories/make-model nodes.
3. **Constraint reconciliation.** Exclude rules prune candidates, *and the system records what it dropped and why.*

6.2 The edit loop — inference-time memory, NOT retraining

For Steffes' trust requirement, retraining is the wrong instrument (collapses auditability; correction lag of hours/days; not reversible; no budget). Corrections live at inference time, by type:

Correction type	Lives in	Takes effect
"tractor should include X" / alias fix	Override/alias table (existing admin store), promoted to a hard constraint	Immediately, deterministically, next query
"the category definition is too broad"	Natural-language category description, re-embedded	Next query, that node only
"this comp / episode was wrong"	ReasoningBank episode with negative gain	Next <i>similar</i> query

The highest-value corrections collapse into the **deterministic alias/override layer the admin console already has** — the PoC feeds the existing mechanism and lets the ReasoningBank remember the correction so it generalizes to paraphrases.

6.3 ReasoningBank (self-improvement engine)

One episode per resolved query: `query_embedding`, `normalized_query`, `category_mapping`, `included[]`, `excluded[]` (with reasons), `outcome`, `correction_delta`, `trust_weight`, `provenance`, `reversible_handle`. On the next query, nearest-neighbor lookup; a high-trust, close episode → reuse mapping (skip the proximity classifier). Per-user memory = a partition key.

Stability vs adaptability (roadmap math, stubbed in PoC): trust updates Kalman-style — $\text{trust_new} = \text{trust_old} + K \cdot (\text{observed} - \text{trust_old})$, K low under noisy/contradictory feedback, high under consistent feedback. A single thumbs-down never rewrites an established mapping. **Hard-constraint overrides are NOT subject to gain decay — an admin rule is law.** In the PoC, trust is hardcoded (admin correction = 1.0, proximity episode = fixed); the gain controller is documented as the scale-up path.

6.4 Determinism & auditability (non-negotiable for Steffes)

- Hard constraints are deterministic and total; the probabilistic layer only fills the gaps the admin hasn't pinned.
- Every classification emits a **replayable receipt**; ReasoningBank episodes are versioned (replay "as of" a date).
- Every self-improvement is a **named, diffable, reversible artifact** ("why did the answer change Tue→Thu?" → diff the bank).
- The action gate **refuses to act below the confidence floor** and logs the bet.
- The sensitive-column blocklist rides this same gate as a hard constraint (a classification needing a blocked column fails validation before action).

7. Azure / PIM Permissions Matrix INTERNAL/BUILD

Context: Kadima personnel access the Steffes subscription **via PIM (just-in-time, time-boxed activation)**. This section is what the Steffes subscription/Entra admin must configure. See §13 for the Terraform-specific permissions.

7.1 Task → Role → Scope

#	Task	Built-in role(s)	Scope	Plane
1	Provision Postgres Flexible Server	Azure Database for PostgreSQL Flexible Server Contributor (fallback Contributor)	target RG	Control
1c	Run DDL / <code>CREATE EXTENSION vector</code> / GRANTS	Entra admin on the server (NOT an RBAC role) — assign Lot Genius Admins group or Philippe's UPN	server	Data
2	Create/configure Azure AI Foundry account + project	Azure AI Account Owner + Cognitive Services Contributor ; day-2 Azure AI Developer	account @ RG; AI Developer @ project	Control + data
2b	Agent runs models / threads	Azure AI Developer @ project + Cognitive Services User on the AOAI/AI resource	project + AOAI	Data
3	Read Synapse curated views	Reader on workspace (control) + <code>GRANT SELECT</code> / db_datareader in the serverless pool (data)	workspace + DB	Both
4	Read/write Blob (tuning + embeddings staging)	Storage Blob Data Contributor (Contributor on the account does NOT read blob data)	account/container	Data
5	Deploy Container Apps (MCP server + embedding job)	Contributor or Azure Container Apps Contributor ; pull from ACR needs AcrPull on the MI	RG / ACR	Control + data
6	Manage embeddings / model deployments (AOAI)	Cognitive Services Contributor (deploy) + Cognitive Services OpenAI User (runtime, on the calling MI)	AOAI account	Control + data
7	Wire managed identities (role assignments)	Role Based Access Control Administrator (preferred) or User Access Administrator	target RG	Control (IAM)
8	Key Vault for secrets	Key Vault Administrator (RBAC mode) or Contributor + access policy	Key Vault	Control + data
9	App registrations / group / admin consent	NOT subscription RBAC — see §7.3	directory	Directory

7.2 Persistent vs PIM-eligible + traps

- **PIM-eligible (deploy-time bursts, activate per session, 8h window):** #1, #2, #5, #6, #7, #8.
- **Persistent (always-on):** the *workload's* runtime data-plane roles on the MCP server / job managed identity (Storage Blob Data, Cognitive Services OpenAI User, Cognitive Services User); plus Philippe's **Reader at subscription**. Workload roles must NOT be PIM-eligible on a service principal — they won't auto-activate and the app breaks.
- **Traps:** (1) **PIM not activated = Terraform/CLI 403 mid-apply** — activate before `plan`, request an 8h window, line up any approver. (2) **Control-plane Contributor ≠ data-plane** in three places — Postgres (need Entra admin to log in), Synapse (need `GRANT SELECT`), Storage (need a Blob Data role). (3) **PIM activation does not refresh an existing token** — re-login after activating. (4) **Foundry project RBAC propagation lag** 5–10 min (expect transient

401s — wait, don't debug code). (5) `azure.extensions` **allowlist** must include `VECTOR` (server param) *before* `CREATE EXTENSION vector` succeeds. (6) **PIM-for-Groups** on Lot Genius Admins → double activation for any data-plane path through that group; prefer a **permanent** membership for Philippe.

7.3 Entra directory-level (separate from subscription RBAC)

Kadima likely lacks these; the client's **Global Administrator / Privileged Role Administrator** must do them: - **Application Administrator** — create/update app registrations + grant **admin consent** (the wall for Teams / M365 Copilot publishing). - **Groups Administrator** or make Philippe **Owner of Lot Genius Admins** (cheapest, object-level, no directory role). - **Teams Administrator** — Teams app catalog publish.

7.4 Minimum-friction "pre-configure ONCE" ask to the client admin

1. Persistent **Reader @ subscription** for Philippe.
2. Set **Lot Genius Admins** group as **Entra admin on the Postgres server**.
3. `GRANT SELECT` on the curated Synapse views to that group (one SQL statement).
4. Make Philippe **Owner of Lot Genius Admins** (and a permanent member).
5. **Admin-consent** the bot/Foundry app registration's Graph + Teams permissions (one click).
6. Pre-assign the workload **managed identity** its runtime data-plane roles (permanent).

8. Identity, PII & Governance (build detail

8.1 Three identity / trust boundaries

1. **User** → **Foundry / Teams**: Microsoft Entra SSO — real per-user identity (use it).
2. **Foundry orchestrator** → **MCP server: managed identity, NOT a shared key**. The PoC fixes the current "one shared function key for all users" smell here — it is one line of config.
3. **MCP server** → **data (pgvector + Synapse)**: managed identity, read-only scoped. **The PII blocklist is enforced server-side, before data leaves the tool** — never trust the LLM to redact.

8.2 Row-level authorization — **DECISION: permissive for the PoC**

Per-user row-level ACLs in Synapse are **out of scope** for the 20-hour PoC. The Entra user identity **is** propagated through the MCP call as a claim so the PII/Security agent can make **coarse allow/deny** decisions; the hook is written now and row-level policy is deferred. *Stated as an explicit decision, not an accident.*

8.3 PII handling

PII improvement over the current column-blocklist is demonstrated on the demo slice: intent/pattern/value-level redaction at the `pii_scrub` tool, server-side, as a hard validation-gate constraint (§6.4). Robust org-wide PII governance is a Phase-2 outcome (§10, Tier 2).

9. IP & Deliverables Boundary INTERNAL/BUILD (with client-facing language noted)

Counsel note: the following is structuring, **not legal advice**. Binding instruments are for counsel to execute. Items marked **TBD** stay out of client hands until cleared.

9.1 The one-line rule

Everything on the client side of the MCP seam is a Deliverable; everything behind it is Kadima Background IP.

9.2 Deliverables — Steffes receives, owns or can operate

- Foundry Agent Service configuration (orchestrator definition, prompts, tool-binding) in Steffes' tenant.
- pgvector schema + embedding-pipeline configuration against the client's Synapse data.
- The **MCP tool contracts/interfaces** (tool list, I/O schemas, endpoint spec) — the *interface*, not the implementation.
- IaC (Terraform) for the client-side footprint.
- Architecture diagrams (marked to show the MCP seam).
- The working demo; knowledge-transfer materials (runbook, how to extend tool contracts).

9.3 Kadima Background IP — used to deliver, retained, NOT transferred

- The agentic AI framework and its orchestration runtime.
- The ~40-agent swarm implementation and topology.
- ReasoningBank and memory mechanisms (AgentDB, learned-pattern store).
- Agent-side logic/prompts/pattern libraries behind the MCP endpoint.
- The **MCP server implementation** (distinct from its published contract).

9.4 GitHub-repo conflict — resolution

The SoW says "source code and configuration assets will be delivered in a GitHub repo." Honored by **scoping the word "source," not breaching it**. - **Repo SHOULD contain:** Foundry config + orchestrator definitions; pgvector schema + embedding config; Terraform/IaC for the client footprint; **MCP tool-interface stubs** (contracts/schemas/endpoint reference); README/runbook/diagrams. - **Repo should NOT contain:** agentic-AI-framework source, swarm implementation, ReasoningBank/memory code, or the MCP server implementation (referenced as a Kadima-operated/ licensed dependency behind the seam — shipped as a built image, **not vendored as source**). - **Action:** add a one-line **Definitions** clause (Deliverables vs Background IP vs Foreground IP) to the SoW as a clarifying **amendment / side letter** — *not a silent reinterpretation*. **TBD — counsel chooses the instrument.**

Suggested Definitions language (counsel to confirm): "*Deliverables*" means the client-side configuration, schema, infrastructure-as-code, and tool-interface contracts delivered to the GitHub repository. "*Background IP*" means Kadima's pre-existing and independently-developed frameworks, tooling, and components used to perform the Services but not delivered; Background IP is accessed by the Deliverables solely through a defined service interface (MCP) and is retained exclusively by Kadima. The repository will contain Deliverables; it will not contain Background IP source.

9.5 Hosting posture — DECISION (client direction)

Hosting will be in **Steffes' Azure subscription**, deployed **only after a mutual usage agreement (Steffes + Kadima)** is in place governing the operation of Kadima Background IP in the client's environment. Because the framework will run in the client's subscription, **IP protection rests on contractual + technical controls rather than tenant separation**: - **Contractual**: the mutual usage agreement + a scoped license grant for the retained framework; Background-IP carve-out in any IP-assignment language. - **Technical**: MCP server shipped as an **opaque/built container image** (no source in the image), registry access controlled; **telemetry/trace verbosity scrubbed** so reasoning chains and prompts do not land in client Log Analytics in plaintext. - **Fallback**: if Steffes declines to expand the engagement, Kadima may provide a **custom, Steffes-specific build of the components under an internal-use-only agreement** (no access to the general framework).

Residual risk (must be stated to stakeholders): a counterparty with owner/root on the host can inspect a running container. The usage agreement + opaque-image + telemetry-scrubbing **reduce** but do not **eliminate** that exposure. This trade was a deliberate client decision in favor of in-subscription hosting and data residency.

9.6 What counsel must paper before Phase 2 (TBD)

1. **MSA + IP clauses** with a Background-IP carve-out and a Foreground/Deliverables assignment split (most important).
2. **License grant** for the retained framework (covers in-subscription operation).
3. **NDA** covering Phase 2 discussions before any agentic AI platform detail is shared.
4. (Optional) a PoC **warranty/limitation** scoping it as proof-of-concept, not production-warranted.

Verify before any binding doc: exact legal entity strings — "**Kadima Consulting**" and "**Steffes Group, Inc.**" (note the comma).

10. Phase 2 — Agentic AI Platform Options ● CLIENT-SHAREABLE (brochure altitude, no IP mechanics)

Phase 2 — Optional Future Direction (non-binding). *Beyond this Proof of Concept, Kadima Consulting offers additional capabilities and tooling — referred to collectively as Kadima's agentic AI platform — that may be available to Steffes Group under a separate future engagement. These capabilities are described here at a conceptual level only and are illustrative of potential directions, not commitments. Any Phase 2 engagement, including its scope, deliverables, commercial terms, and the terms under which any Kadima tooling or framework would be made available, would be defined in a separate written agreement. Nothing in this document constitutes an offer, a license, a price quote, or a commitment by either party to proceed, and no proprietary or confidential details of Kadima's underlying technology are disclosed herein.*

10.1 The strategic hook

A successful PoC does not merely *answer* the comps question — it proves Steffes owns an asset it has never been able to use: **the institutional memory of every lot it has ever sold**. Five hundred thousand transactions become a living, queryable, self-sharpening pricing intelligence available to every appraiser in seconds. The comps agent is the beachhead, not the destination.

10.2 Capability tiers (a staircase, not a buffet — outcomes only)

- **Tier 1 — Self-Improving Institutional Memory.** Every appraiser correction makes the next answer better, automatically, organization-wide. Your best appraiser's judgment becomes the floor for everyone, and it compounds — value that stays with Steffes.
- **Tier 2 — Governed, PII-Aware Knowledge Access Across the Org.** Extend trusted retrieval beyond comps to wider organizational data, with role-aware boundaries and the auditability leadership needs to deploy it broadly.
- **Tier 3 — Agentic Pricing & Trend Intelligence.** From "find comparable lots" to "tell me what's happening" — seasonal movement, category drift, regional demand, reserve guidance, surfaced proactively.
- **Tier 4 — Reach Beyond the Vault.** Extend the same intelligence to live/upcoming lots and enrich with third-party market signals (e.g. MachineryPete-class external comps).
- *(Adjacent, sequenced when the platform decision lands: a Fabric-native path so this intelligence rides Steffes' chosen Microsoft data strategy.)*

10.3 The recommended next step

Tier 1 (Self-Improving Institutional Memory) is the natural next move: the PoC already ships a human-editable correction loop, so the question writes itself — "does it remember tomorrow? does the next appraiser get the benefit?" It is the lowest-effort step (the loop already exists) and it makes every subsequent tier more valuable.

11. Risks & Open Decisions

#	Risk / Decision	Owner	Status
R1	Data-quality bet (sinks the PoC if wrong): does a chosen slice contain enough clean structured signal (make/model/year/spec/condition + hammer price) for "comps" to be genuinely comparable, and can PII be cleanly separated at row level?	Kadima + Steffes SME	De-risk in first 2 hours — pull 50 real lots, eyeball before coding. Pivot the slice if not.
R2	In-subscription hosting residual IP exposure (root-on-host)	Kadima / counsel	Accepted with mutual usage agreement + opaque image + telemetry scrub (§9.5)
R3	SoW "deliver all source" vs IP retention	Counsel	Definitions amendment/side letter — TBD instrument
R4	PIM activation friction blocking Terraform mid-apply	Steffes admin	Pre-configure §7.4; activate 8h windows before <code>plan</code>
R5	Embedding model / dimensionality choice; index immutability	Kadima	Workshop decision; <code>text-embedding-3-large</code> default, lock per index
R6	Synapse deprecation (platform EOL)	Steffes	Out of PoC scope; Fabric path = roadmap (§10)
D1	Legal entity strings verification	Philippe	TBD — confirm
D2	Demo slice selection (combines + tractors?)	Philippe + SME	Confirm at workshop

12. Evaluation Plan & Demo Script

- **Gold set:** ~20–30 representative questions (comps, trends, descriptions, "no-good-comp" cases) with human-blessed answers, frozen at workshop end.
- **Metrics:** SC1 comps-usable rate (appraiser rating), SC2 fabrication rate (every number must cite a Lot ID), SC3 latency p50/p95, SC4 admin-edit-to-live-effect time.
- **Demo script (live, Teams / M365 Copilot):** 1. Appraiser: "Show me 5 comps for a 2023 John Deere X9 1100" → 5 comparable lots, prices, Lot-ID citations, < 10s. 2. Appraiser: a deliberately sparse query → graceful "I don't have good comps" (no fabrication). 3. Admin: open the **classification receipt**, correct a mis-categorization, re-ask → corrected result, same session. 4. (If time) Admin: mark a value sensitive → redacted on next query (PII secondary anchor). 5. (If time) Appraiser: YoY combine trend with row count.

13. Azure Resource Inventory & Terraform Delivery INTERNAL/BUILD

13.1 Resources Terraform will create

Target resource group: `rg-steffes-copilot` (existing — SoW grants Contributor) or a dedicated `rg-steffes-lotgenius-poc` for clean teardown (recommended if permitted). Existing assets reused are marked *(existing)*.

#	Resource (Azure type)	Purpose
1	<code>azurerm_postgresql_flexible_server</code>	Managed Postgres for the vector store
2	<code>azurerm_postgresql_flexible_server_database</code>	Application DB
3	<code>azurerm_postgresql_flexible_server_configuration</code> (<code>azure.extensions = VECTOR</code>)	Allowlist pgvector before <code>CREATE EXTENSION</code>
4	<code>azurerm_postgresql_flexible_server_active_directory_administrator</code>	Entra admin (Lot Genius Admins group) for DDL
5	<code>azurerm_postgresql_flexible_server_firewall_rule</code> / VNet integration	Network access (public+firewall for PoC, or private endpoint)
6	<code>azurerm_ai_foundry</code> (Azure AI / Cognitive Services account)	Foundry account
7	<code>azurerm_ai_foundry_project</code>	Foundry project hosting the agent
8	<code>azurerm_cognitive_deployment</code> xN	Model deployments: <code>text-embedding-3-large</code> , intent chat model, MAI-Thinking-1
9	<code>azurerm_container_app_environment</code>	Hosting env for the MCP server + jobs
10	<code>azurerm_container_app</code> (MCP server)	The MCP server — Kadima agentic AI framework (built image)
11	<code>azurerm_container_app_job</code> (embedding/ETL)	Synapse → embeddings → pgvector pipeline

#	Resource (Azure type)	Purpose
12	<code>azurerm_container_registry</code> (or existing)	Holds the MCP server image (<code>AcrPull</code> to the MI)
13	<code>azurerm_user_assigned_identity</code>	Workload identity for MCP server + job
14	<code>azurerm_key_vault</code> + secrets	Connection strings, keys, cross-tenant creds
15	<code>azurerm_log_analytics_workspace</code> / <code>azurerm_application_insights</code> (or existing)	Monitoring (telemetry verbosity scrubbed — §9.5)
16	<code>azurerm_role_assignment</code> xN	Wire MIs → AOAI (<code>Cognitive Services OpenAI User</code>), Storage (<code>Storage Blob Data</code>), ACR (<code>AcrPull</code>), Key Vault
17	<code>azurerm_storage_container</code> (existing storage)	Embeddings staging / tuning container
—	(existing) Synapse Serverless workspace + curated views	Data source — read-only, NOT created
—	(existing) Storage account <code>tuning/</code>	Tuning assets — reused
—	Decommission: the legacy <code>/api/ask</code> Azure Function (LLM-SQL)	Replaced by parameterized SQL templates — retire post-cutover

13.2 Terraform state backend (bootstrap — create first, once)

- `azurerm_storage_account` + `azurerm_storage_container` (e.g. `tfstate`), blob versioning on. The Terraform identity needs **Storage Blob Data Contributor** on this container. Keep state outside the resource groups Terraform manages.

13.3 Permissions required for Terraform to run end-to-end

The identity running Terraform (Philippe via PIM, or a dedicated Terraform service principal with OIDC):

Subscription / resource-group RBAC (control plane): - **Contributor** @ target RG (covers creation of items 1–17), **OR** the granular set: *Azure Database for PostgreSQL Flexible Server Contributor*, *Azure AI Account Owner + Cognitive Services Contributor*, *Azure Container Apps Contributor*, *AcrPush/Contributor (registry)*, *Key Vault Administrator*, *Monitoring Contributor*, *Storage Account Contributor*. - **Role Based Access Control Administrator** (or User Access Administrator) @ target RG — **REQUIRED:** Terraform creates `azurerm_role_assignment` (item 16). Without it the apply fails at the IAM steps. (Classic Terraform-on-Azure failure.) - **Reader** @ subscription (persistent) + **resource-provider registration** for `Microsoft.DBforPostgreSQL`, `Microsoft.App`, `Microsoft.CognitiveServices`, `Microsoft.ContainerRegistry`, `Microsoft.KeyVault`, `Microsoft.OperationalInsights` — **Contributor** @ subscription registers them automatically; otherwise pre-register.

Storage data plane: **Storage Blob Data Contributor** on the `tfstate` container (state) and on any blob container Terraform writes.

Entra directory (azuread provider, if Terraform manages app regs): **Application Administrator** / `Application.ReadWrite.All` — **separate from subscription RBAC;** Kadima likely lacks this → client admin pre-creates app registrations, or grants it (§7.3).

Out-of-band steps Terraform CANNOT do via ARM (document + script as `null_resource` /local-exec or manual runbook): - `CREATE EXTENSION vector`; and any `GRANT` inside Postgres (needs an Entra-admin SQL connection — chicken-and-egg with PIM; prefer Philippe as a permanent Entra-admin/member). - `GRANT SELECT / db_datareader` on the Synapse curated views (issued by an existing Synapse SQL admin). - Admin-consent for Graph/Teams permissions (Global Admin).

PIM timing for Terraform: activate the required eligible roles **before** `terraform plan`, with an **8h** window (a Foundry + Postgres + Container Apps apply can exceed 1h); a long apply that crosses the activation expiry causes split state. Re-`az login` after activating (PIM does not refresh an existing token).

Appendix A — Source Materials Reviewed

- `Lot_Genius_PoC_SoW.docx` — the governing Statement of Work (Kadima ↔ Steffes).
- `Lot_Genius_20260506.docx` — current-state architecture reference.
- `LotGenius2-0.docx` — Kadima memo (proposed architectures, Options 1–3).
- `Copilot_Agent.docx` / `meeting1.txt` — discovery meeting transcripts (Sean Todd, Marc Farron, John Mendes, Philippe Richard).

Appendix B — Research Grounding (agentic RAG self-improvement + HITL classification)

Design choices in §6 are grounded in current literature: Self-RAG and RAG-Critic (self-reflection / critic-guided correction), the Agentic RAG survey (reflection/planning/tool-use/multi-agent taxonomy), PatchRAG / Feedback-Adaptation (inference-time correction without retraining; "correction lag" + "post-feedback performance"), GAM-RAG (training-free evolving retrieval memory with a Kalman-style gain rule for stability vs adaptability), and the human-in-the-loop classification line (feedback on *features/rules* not just instances; natural-language category refinement handling unseen categories without retraining; continual-learning HITL). These support the core decision: **inference-time memory over fine-tuning** for an auditable, reversible, fast-correcting system.

End of PRD v1.0 (draft). Items marked TBD require Philippe / counsel confirmation before client circulation.